Watchdog Robot - “Johnny Boy The Italian”

Machine Minds: Final Group Project

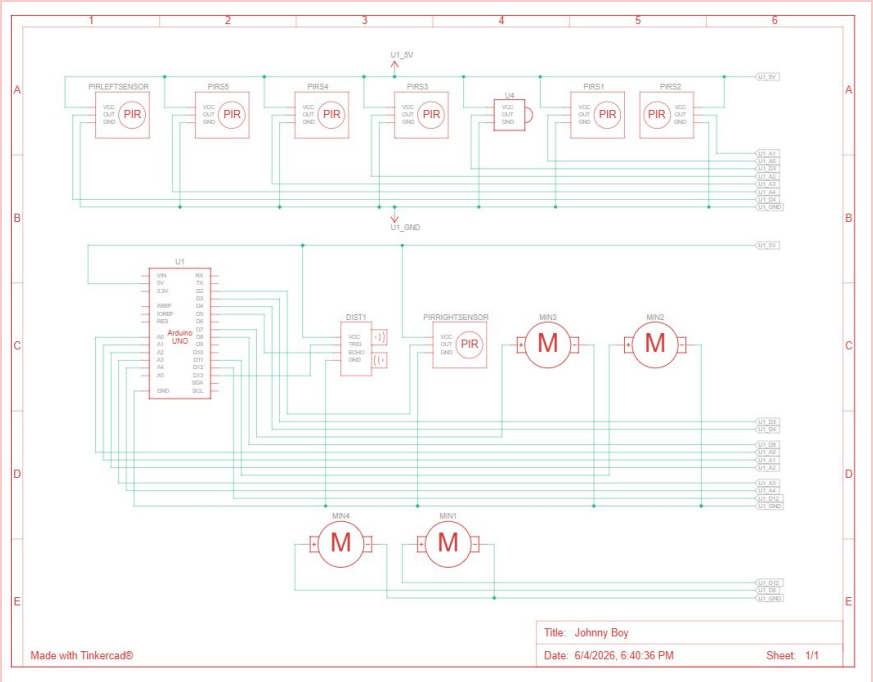
By Lili, Lincoln, Noa, Evan, Ben, and Jaxon

Introduction (Jaxon)

- The real-world problem our robot addresses is security guards who fall asleep on the job (our robot won't do that!)
- Objective:
  - **1.** to patrol a doorway space
  - **2.** detect "threats" based on passing movement
  - **3.** "attack" them

Robot Design and Features (Evan)

- Walk through our design: chosen sensors, wiring, motor layout, how the pieces fit together…
- Guard Dog - Patrols, Chases
- Line tracker to allow it to Patrol
- Ultrasonic to identify intruders to Chase
- PIR to not hit anything while Patrolling



Code Structure - Pins, Functions

```
#include <IRremote.hpp>
#define ENA 9
#define IN1 12
#define IN2 11
#define ENB 6
#define IN3 7
#define IN4 8


// ultrasonic and LED pins
const int trigPin = 13;
const int echoPin = 5;

// 5-channel IR line tracking sensor pins
const int s1 = A0;  // far left
const int s2 = A1;  // mid left
const int s3 = A2;  // center
const int s4 = A3;  // mid right
const int s5 = A4;  // far right

const int irPin = 3;
const int leftSensor = 4;
const int rightSensor = 2;

int SPEED = 110;
int TURN_SPEED = 120;
int SHARP_TURN_SPEED = 150;

// State constants
const int STATE_CENTER = 0;
const int STATE_LEFT = 1;
const int STATE_SHARP_LEFT = 2;
const int STATE_RIGHT = 3;
const int STATE_SHARP_RIGHT = 4;
const int STATE_LOST = 5;

// declare currentState and lastDirection global variables
int currentState = STATE_CENTER;
int lastDirection = 0;  // -1 = left, 0 = center, +1 = right

int displayMode = 0;
bool stopState = false;

const unsigned long BUTTON_1 = 0xBA45FF00;
const unsigned long BUTTON_2 = 0xB946FF00;
const unsigned long BUTTON_3 = 0xB847FF00;
```

```
void goForward() {
  analogWrite(ENA, SPEED);
  analogWrite(ENB, SPEED);
  // set motor direction for forward
  //   IN1=HIGH IN2=LOW   IN3=HIGH IN4=LOW
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void turnLeft() {
  analogWrite(ENA, TURN_SPEED);
  analogWrite(ENB, TURN_SPEED);
  // set motor direction for soft left turn (only right motor moves)
  //   IN1=LOW  IN2=LOW   IN3=HIGH IN4=LOW
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void turnRight() {
  analogWrite(ENA, TURN_SPEED);
  analogWrite(ENB, TURN_SPEED);
  //  set motor direction for soft right turn (only left motor moves)
  //   IN1=HIGH IN2=LOW   IN3=LOW  IN4=LOW
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

void sharpTurnLeft() {
  analogWrite(ENA, SHARP_TURN_SPEED);
  analogWrite(ENB, SHARP_TURN_SPEED);
  // set motor direction for sharp left turn (pivot left)
  //   IN1=LOW  IN2=HIGH  IN3=HIGH IN4=LOW
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}
```

```
void sharpTurnRight() {
  analogWrite(ENA, SHARP_TURN_SPEED);
  analogWrite(ENB, SHARP_TURN_SPEED);
  // set motor direction for sharp right turn (pivot right)
  //   IN1=HIGH IN2=LOW   IN3=LOW  IN4=HIGH
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void stopRobot() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  //set motor direction for stop (all LOW)
  //   IN1=LOW  IN2=LOW   IN3=LOW  IN4=LOW
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

// Wait ms milliseconds while still checking the IR receiver.
// If a button is pressed during the wait, update displayMode and return true.
// Returns false if the full wait time elapses with no button press.
bool delayCheckIR(unsigned long ms) {
  unsigned long start = millis();
  while (millis() - start < ms) {
    if (IrReceiver.decode()) {
      unsigned long buttonCode = IrReceiver.decodedIRData.decodedRawData;
      if (buttonCode == BUTTON_1) displayMode = 1;
      else if (buttonCode == BUTTON_2) displayMode = 2;
      else if (buttonCode == BUTTON_3) displayMode = 3;
      IrReceiver.resume();
      return true;
    }
  }
  return false;
}
```

Code Structure - Patrol

```
// Once robot reaches end of path,
// turns until it detects the line again continues on patrol

void turnAround() {
  int r1 = digitalRead(s1);
  int r2 = digitalRead(s2);
  int r3 = digitalRead(s3);
  int r4 = digitalRead(s4);
  int r5 = digitalRead(s5);
  stopRobot();
  if (r1 == HIGH && r2 == HIGH && r3 == HIGH && r4 == HIGH && r5 == HIGH) {
    sharpTurnLeft();
  } else if (r3 == LOW) {
    stopRobot();
  }
}
```

```
// Follows A Set Path via Line Tracker
void backAndForth() {
  // ultrasonic
  unsigned long duration;
  int distanceCm;

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);

  digitalWrite(trigPin, LOW);

  // Read echo Pulse
  duration = pulseIn(echoPin, HIGH);

  // convert time to distance
  distanceCm = duration / 58;

  int r1 = digitalRead(s1);
  int r2 = digitalRead(s2);
  int r3 = digitalRead(s3);
  int r4 = digitalRead(s4);
  int r5 = digitalRead(s5);
  // sets priority fo sensors
  if (r3 == LOW) {
    currentState = STATE_CENTER;
    lastDirection = 0;
  } else if (r2 == LOW) {
    currentState = STATE_LEFT;
    lastDirection = -1;
  } else if (r1 == LOW) {
    currentState = STATE_SHARP_LEFT;
    lastDirection = -1;
  } else if (r4 == LOW) {
    currentState = STATE_RIGHT;
    lastDirection = 1;
  } else if (r5 == LOW) {
    currentState = STATE_SHARP_RIGHT;
    lastDirection = 1;
  } else if (r1 == HIGH && r2 == HIGH && r3 == HIGH && r4 == HIGH && r5 == HIGH) {
    currentState = STATE_LOST;
  }
```

```
  // switch tells robot what to do based on current State
  switch (currentState) {
    case STATE_CENTER:
      goForward();
      if (distanceCm < 25) {
        stopRobot();
      }
      break;
    case STATE_LEFT:
      turnRight();
      if (distanceCm < 25) {
        stopRobot();
      }
      break;
    case STATE_SHARP_LEFT:
      sharpTurnRight();
      if (distanceCm < 25) {
        stopRobot();
      }
      break;
    case STATE_RIGHT:
      turnLeft();
      if (distanceCm < 25) {
        stopRobot();
      }
      break;
    case STATE_SHARP_RIGHT:
      sharpTurnLeft();
      if (distanceCm < 25) {
        stopRobot();
      }
      break;

    case STATE_LOST:
      if (lastDirection == -1) {
        sharpTurnLeft();
      } else if (lastDirection == 1) {
        sharpTurnRight();
        // if bot reaches end of set line will turn around
        // and continue back the other way
      } else if (lastDirection == 0) {
        turnAround();
      }
      break;
  }
  delay(5);
}
```

Code Structure - Attack, Setup, Loop

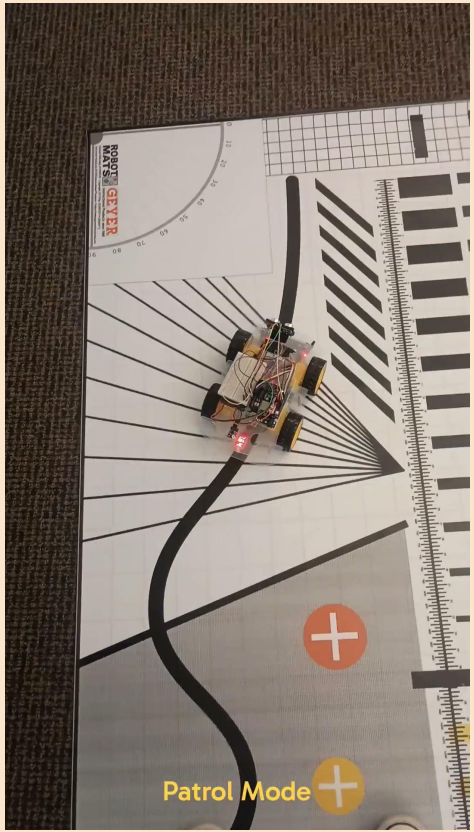
```
// Attack Mode: Follows intruder and blares an alarm
void attackMode() {
  // ultrasonic
  unsigned long duration;
  int distanceCm;

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);

  digitalWrite(trigPin, LOW);

  // Read echo Pulse
  duration = pulseIn(echoPin, HIGH);

  // convert time to distance
  distanceCm = duration / 58;


  SPEED = 90;
  int IRLeft = digitalRead(leftSensor);
  int IRRight = digitalRead(rightSensor);

  if (IRLeft == LOW && IRRight == LOW) {
    stopRobot();
    if (distanceCm < 25) {
      goForward();
    }
  } else if (IRLeft == HIGH && IRRight == HIGH) {
    sharpTurnLeft();
    if (distanceCm < 25) {
      goForward();
    }
  } else if (IRLeft == LOW && IRRight == HIGH) {
    turnRight();
    if (distanceCm < 25) {
      goForward();
    }
  } else if (IRLeft == HIGH && IRRight == LOW) {
    turnLeft();
    if (distanceCm < 25) {
      goForward();
    }
  }
}
```

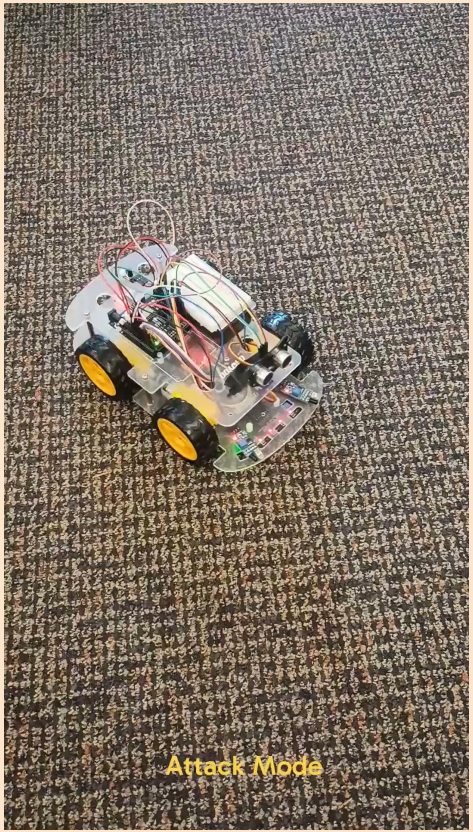
```
void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(leftSensor, INPUT);
  pinMode(rightSensor, INPUT);
  IrReceiver.begin(irPin, ENABLE_LED_FEEDBACK);

  // ultrasonic
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}

void loop() {
  // Top-of-loop IR check — handles button presses between actions
  if (IrReceiver.decode()) {
    unsigned long buttonCode = IrReceiver.decodedIRData.decodedRawData;
    if (buttonCode == BUTTON_1) displayMode = 1;
    else if (buttonCode == BUTTON_2) displayMode = 2;
    else if (buttonCode == BUTTON_3) displayMode = 3;
    IrReceiver.resume();
  }



  switch (displayMode) {
    case 1:
      backAndForth();
      break;
    case 2:
      attackMode();
      break;
    case 3:
      stopRobot();
      break;
  }
}
```

Troubleshooting (Noa)

- Challenges we faced included the buzzer not working right
- Buzzer kept interrupting our previous code, ignoring IR remote inputs (we decided to not include it in the end…)
- Struggling to get the line tracker set up
- We were able to stick with our initial design, the only design tradeoff being removing the buzzer
- If given more time, our group could work on getting the buzzer fully functional

Demonstration (Lincoln)





What Each Member Contributed (Ben)

Lili: frame-building, coding, troubleshooting, etc.

Lincoln: frame-building, coding, troubleshooting, etc.

Noa: frame-building, slideshow creation, etc.

Evan: wiring, slideshow

Ben: frame-building,slideshow

Jaxon: troubleshooting, slideshow creation

The End!

Thank you for your time :)